Instructor and mentor guide

MechaRAMS at the YMCA Eight one hour robotics sessions for grades 5 and 6

This manual lets an educator prepare high school juniors and seniors to lead the pilot. Start with the setup and workshop pages, then use each week's two-page plan together with its slides, student worksheet, and complete Python project. The slide deck supports discussion; the manual controls timing and kit rotation.

The plan in one paragraph

12 children in grades 5–6 form four teams of three: T1/T2 share Kit A and T3/T4 share Kit B. Four student instructors cover two robot stations and two build tables; a fifth or sixth can float. The adult lead oversees the room and YMCA procedures. Bring four available laptops: two remain connected to robots and two show build references. Printed build cards make the build activity work with two laptops too.

What is ready and what needs local validation

The package supplies lesson sequences, student activities, presentations with notes, and original SPIKE App 3 Python starters. Code syntax and key logic can be checked offline. The real motors, sensors, app version, cables, chassis direction, surface, and thresholds must be tested locally. Complete the acceptance checklist before student use. Do not advertise a hardware-tested curriculum until that rehearsal is complete.

How to use the files

Open README.md on GitHub or in the folder. Each week has a numbered PowerPoint and a Python file. Paste the complete Python file into a SPIKE Prime Python project; .py files are source text, not native SPIKE project bundles. Keep the student edits inside student_recipe, except for the week 6 named function and threshold. Export saved projects through the app and also save a text copy of the final recipe.

Teaching stance

Ask "What did you expect?" and "What evidence would help?" Give students a little thinking time. Offer one hint, then return the keyboard. Praise a careful test or a clear explanation. Avoid speed-based rankings. A failed test with a sound explanation is useful work.

Navigation

Setup and launch pages come first, followed by the 120-minute mentor workshop, the weekly rotation schedule, eight lesson plans, assessment keys, Word Blocks guidance, and a hardware acceptance record. The proposal document contains the research links and planning budget.

A launch checklist

- Confirm eight dates, a make-up plan, room access, storage between weeks, and who will charge the hubs. Confirm the actual YMCA site and coordinator before publishing details.
- Confirm registration, emergency contacts, pickup, accessibility needs, visitor arrangements, and photo permissions through YMCA staff. High school mentors do not manage private family records.
- Confirm two functioning SPIKE Prime hubs, batteries, at least two wheel motors per kit, wheels, pins and beams, one distance sensor and one color sensor per kit, chargers, and data cables. Inventory both bins separately.
- Confirm two compatible laptops at minimum or four preferably. Install the official current SPIKE App, test Python and project save/export, and record the app and hub firmware versions. Check account and installation permissions in advance.
- Prepare one assembled driving base per kit using the supplied official PDF. Trace cables and apply this curriculum's port map. Photograph the final working configuration for your mentors.
- Run the smoke test, forward movement, pivot, distance readings, and reflection readings on both robots. Rehearse each week's mission on the actual floor. Keep known-good project copies.
- Assign the adult lead, two robot mentors and two build mentors. Additional available student instructors float or record class-level evidence. Train everyone on the actual stop, reset and troubleshooting routines.
- Print worksheets and code cards. Label team folders T1–T4 and hubs YMCA A and YMCA B. Prepare two non-overlapping lanes and an off-robot work area.

A practical session routine

Arrive 20–30 minutes early. Charge-check both hubs, open that week's files, and do one test run in each lane. At the end, reserve six minutes to save work and count parts. After students leave, allow about 15 minutes to back up files, charge equipment, record attendance totals, and note a change for next week.

When a kit is unavailable

If only one robot works, tell the adult lead. Use four eight-minute team turns within 32 minutes and rotate the three jobs in roughly equal thirds. Other teams build and hand-test with mentors. Reduce the robot goal to one recorded trial and use the remaining practice time for recovery. This is a reduced-access contingency; restore both kits before the next class.

Build and connect the two robots

Use 06_References/LEGO_Practice_Driving_Base.pdf. It contains 37 pages and steps 1–34. Keep the assembled bases for the entire program. The auxiliary attachment motor in the official base is not needed for our core missions and can remain electrically disconnected.

1. Sort each kit separately. Follow the official numbered build steps in order. Check that wheel axles move freely and that the chassis sits level. Do not take parts from the other kit without recording the transfer.

2. Choose and label the front of the robot. Follow the cable from the motor that drives the LEFT wheel, viewed from behind the robot facing forward. Plug that cable into A. Plug the RIGHT wheel motor into B. This mapping overrides any different cable locations shown in the original PDF.

3. Keep any auxiliary attachment motor cable unplugged and secure it clear of wheels. C is reserved for the distance sensor in week 4. D is reserved for the downward color sensor in week 5. E and F are unused in the core plan.

4. Run 00_mentor_smoke_test.py with wheels raised. Confirm the two wheels cooperate for forward motion. Then place the robot on the floor for a short test. If it reverses or spins, stop and check front label, physical assembly, and left/right cable mapping. Resolve the configuration before class.

5. Before week 4, use spare beams and pins to make a rigid bracket that holds the distance sensor facing straight ahead, level with a broad cardboard target. Leave its two openings clear. Plug it into C. Record a front and side photo of the successful mounting.

6. Before week 5, mount the color sensor so its lens faces straight down and sits about 1–2 cm above the surface as a starting point. Adjust height until white and dark readings separate clearly. Plug it into D. Keep it clear of the wheel path and later plow.

7. For week 6, use unused front beam holes to attach a passive bumper or U-shaped guide with two independent pin connections. Aim for a small ground clearance so it can push the cube without dragging. Reserve parts for all four teams and test a quick attachment swap.

Sensor bracket and plow geometry depend on the available spare pieces and local build. These mounting instructions describe the required function, not a LEGO-verified numbered attachment build. The educator must make and photograph a working example before mentors teach it.

Port	Device	Check
A	Left wheel motor	Trace cable from physical left wheel
B	Right wheel motor	Trace cable from physical right wheel
C	Distance sensor	Forward, clear of attachment
D	Color sensor	Downward, stable contrast
E and F	Unused	Keep cable paths clear

Prepare the Python workspace

1. Download the official SPIKE App from source [2] in the proposal. Open SPIKE Prime, create a new Python project, and connect one named hub. Use a data-capable USB cable for initial setup when available. Perform firmware updates before the class, not during it.

2. Open the supplied .py file in a text editor, copy the complete contents, and replace the app's initial sample code with it. The imports refer to modules on the SPIKE hub. Running this file in ordinary desktop Python will not control the robot.

3. Run the mentor smoke test first. Confirm the app's stop control and the hub's program-stop action on the actual device. Show mentors how to stop without grabbing a moving mechanism.

4. Save one known-good native project per week. Save team work as W02_T1_delivery. Keep separate T1/T2 projects on the Kit A laptop and T3/T4 on Kit B. The two build-reference laptops do not connect to the hubs. Save before switching teams.

5. Show the student_recipe function near the top first. The support code below supplies drive, turn, signal, park, and seek_marker. Students do not need to type that entire layer. In weeks 4 and 5, reveal the relevant sensor condition and counted loop so the decision is understandable.

6. At each session end, use the app's save/export workflow and copy the student recipe into the team's local folder. For browser use, explicitly download the project. A browser tab is not a durable backup.

Words to teach only when needed

A function is a named action or group of actions. A parameter supplies information to it. A variable names a value. Indented lines belong to a loop, condition, or function. await lets an action finish before the next recipe line continues. async def is the form used for our action functions; concurrent programming is outside this course.

Avoid version mixing

These projects use SPIKE App 3 style modules such as motor_pair and runloop. Older examples beginning with from spike import PrimeHub use the legacy API. Pybricks uses another firmware and API. Do not combine these examples in the same class project. Check current app help if a signature differs from the documentation archive [8–9].

Safety and limits in the teaching code

Movement uses small finite commands. Sensor searches stop after 12 or 24 steps, invalid distance readings stop the search, and main uses finally to stop the motor pair after an exception. These controls reduce runaway behavior but do not replace a clear lane, adult oversight, or an available manual stop. Calibration is still required.

Train the high school mentors

A proposed 120-minute workshop led by the adult educator. Each mentor must demonstrate both robot handling and teaching before leading a pod.

Time	Activity	Readiness evidence
0–10	Purpose, student experience, and YMCA procedures	Mentor identifies who handles supervision and pickup
10–30	Inventory, chassis, ports, connection, stop	Mentor runs and stops the smoke test
30–50	Practice sessions 1–3 in teams	Mentor changes a variable and traces the loop
50–70	Read sensors and test sessions 4–5	Mentor explains mm, reflection, invalid data, and limits
70–85	Build and swap a passive plow	Mentor checks stability, clearance, and sensor visibility
85–100	Teach a five-minute mini lesson to another mentor	Teacher uses prediction and a question before a hint
100–110	Practice failure and role-rotation scenarios	Mentor repairs indentation and prevents keyboard takeover
110–120	Rehearse transitions and capture evidence	Mentor saves correct team file and records an honest outcome

A repeatable coaching conversation

Ask: “What did you expect?” Then: “What did you actually observe?” Then: “Which one thing could you change to test that idea?” Wait briefly for an answer. If needed, point to the relevant line. If that fails, model one edit and have the student explain and make the next edit. Avoid taking over the entire task.

Two rehearsal scenarios

Scenario 1: one student keeps the keyboard. The mentor announces the planned swap and asks the former driver to predict the next result. Scenario 2: the robot hits the movement limit. The mentor asks the team to read the stop reason and check starting distance rather than removing the limit.

Mentor readiness check

Each mentor can connect the correct hub, explain the port map, stop a run, recover a known-good file, teach the current concept in plain language, rotate roles, and identify when to ask the adult lead for help. A mentor who is still learning a sensor can assist an experienced mentor until ready.

Use the hour productively

Minutes	Purpose	What the mentors do
0–5	Welcome and introductions	Week 1: names, mentor introductions and the stop signal. Begin with curiosity and the robot demonstration. Later weeks: a short recall or physical example.
5–10	Robot demonstration	One short demonstration and the hub-light check. Week 1 includes the separate EV3 demo only if ready.
10–14	Code and prediction	Keep the code and prediction on one slide. Read the robot rule aloud. Each team chooses one thing to test.
14–26	First rotation	T1 and T3 use robots. T2 and T4 build. Rotate the three jobs at minutes 18 and 22.
26–38	Second rotation	T2 and T4 use robots. T1 and T3 build. Rotate the three jobs at minutes 30 and 34.
38–44	First retest	T1 and T3 use robots. Rotate jobs at 40 and 42. T2 and T4 finish or improve their builds.
44–50	Second retest	T2 and T4 use robots. Rotate jobs at 46 and 48. T1 and T3 finish or improve their builds.
50–54	Learning and observations	Each child shares one idea or observation within their six-child pod. Use the new reflection prompts.
54–60	Save and reset	Save by team ID, store each team build, count parts, stop robots and follow YMCA pickup procedures.

Three roles at each station

At the robot, the driver edits and starts after “clear.” The navigator reads the step, predicts, positions the stopped robot and payload, fits an approved attachment and checks the lane. The observer watches, measures and records one number, tick or phrase. Rotate all three jobs every four minutes in the first turn and every two minutes in the retest. At the build table, rotate builder, designer/parts manager and tester/reporter on the same timer. Everyone contributes hands-on work.

Two teams in each pod

14–26: T1/T3 use robots and T2/T4 build. 26–38: teams switch stations. 38–44: T1/T3 retest while the other teams finish builds. 44–50: T2/T4 retest. Rotate roles at 18, 22, 30, 34, 40, 42, 46 and 48 minutes. Each team has 18 robot minutes, giving each child about six minutes as driver plus handling, observing and building. Transitions occur within these blocks.

A time-saving rule

After two minutes of technical trouble, use the recovery checks. After five, restore a known-good project or move to the hands-on build and stationary demonstration. Preserve reflection and cleanup by dropping an extension.

If students miss a week

Use a five-minute reentry: partner explains the previous goal, mentor provides the working starter, and the returning student predicts and changes one value. Do not require home installation or lengthy make-up work.

Week 1 Meet your robot

Mission A signal and a first movement

Learning objective Each student changes a short Python message, predicts a short action sequence, and explains that a robot follows the order of its instructions.

Minimum success The team displays its chosen one-letter signal, runs one short forward movement, and all three teammates identify which instruction happened first.

Materials Assembled base, labeled hub, one laptop at each robot station, paper name flag, tape, ruler, team sheets and build cards.

Before students arrive Keep wheels raised for the first signal run. Open W01_signal.py in a SPIKE Prime Python project. Confirm the A/B motor mapping with the mentor smoke test. Welcome children and introduce the mentors. Prior-experience information has already been collected; use no opening quiz or ability rating.

Time	Activity
0–5	Welcome and introductions
5–10	Robot demonstration
10–14	Code and prediction
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt Where could a robot help at a community center? Accept practical or playful ideas. Ask what the robot must be told before it can help.

Demonstration Run a letter followed by a pause and a short move. Point at the matching line as each action occurs. Say: “Python is text that tells the robot what to do. Today you only change the recipe area.”

```
await signal("Y")
await pause(500)
await drive(90)
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 1 facilitation steps

1. Name the hub and locate its ports. Point out the two motors, wheels, rechargeable hub, and display. Keep the chassis assembled. Add a removable paper name flag that clears wheels and ports.

2. Open the supplied project. Find STUDENT RECIPE inside student_recipe. Change Y to one letter chosen by the team. Keep quotation marks. Run with wheels raised, then point to the changed line.

3. Place the robot at a taped start in a clear lane. Navigator checks that the lane is empty. Driver starts the program. Observe which action happens first and mark the endpoint.

4. Switch roles. Change the pause from 500 to 1000 milliseconds. Predict the difference and run again. Keep drive at 90 wheel degrees.

5. During the final six-minute turn, reorder signal and pause or choose a different letter. Each student explains one line without the mentor answering for them.

Build activity A removable robot mascot. Make a name flag or face that stays upright and attaches without covering a wheel, port, button or cable. Choose a team name and lay out a lightweight mast or face. Build a wide or braced support using the mounting template. Gently tilt and tap the model by hand. Improve one weak connection. Test: Stands after a gentle tap; stays inside the clearance template; no loose parts. Fit the approved mascot to the stopped base for the retest. The first robot teams use a mentor example or the bare base.

Support Mentor highlights only the three recipe lines. Students may point or dictate while their teammate types, then swap the explanation role. Accept a correct light sequence if movement is delayed.

Optional extension Add a second signal and explain why its location in the sequence matters. No speed contest.

Troubleshooting No letter: check hub connection and that the program started. SyntaxError: check the quotation marks and indentation. Wrong motion: stop and check the A/B cables and base orientation before changing code.

Word Blocks equivalent New Word Blocks project: when program starts; display chosen text on hub; wait 0.5 seconds; move straight for 0.25 rotations at a mentor-tested slow speed. Set movement motors to A and B. Compare the same action order.

Evidence to keep Record the letter, pause value, predicted order, and observed order. Individual exit: “A program is ___, and the line I changed made ___ happen.”

Expected reasoning Show Y, wait, then drive. A longer pause changes waiting time, not the drive distance. A signal-only run still demonstrates sequence.

Week 2 Delivery to a target

Mission Wheel turns predict forward distance

Learning objective Each child explains how a motor turns a wheel, predicts distance from wheel degrees, and estimates wheel degrees for a target distance using circumference.

Minimum success The team makes three recorded trials and attempts to stop the fixed chassis mark within 5 cm of a 40 cm target. A justified adjustment counts even if precision remains imperfect.

Materials Base, rulers, non-stretch string/paper strips, calculator, a fixed chassis-reference mark, 40 cm target, wheel reference card and trial log.

Before students arrive Measure each kit's actual wheel diameter and circumference before class. Check that the drive wheels attach directly to the motor outputs. Mark two clear floor lanes with a 40 cm target measured from one fixed chassis reference. Rehearse 180, 360 and the calculated target value at SPEED = 200. Keep the 0–1080 wheel-degree limit.

Time	Activity
0–1	Welcome, delivery goal and brief recap
1–11	Motor, turns, rolling, circumference and both conversions
11–14	Code, explicit predictions and short demonstration
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt A delivery cart must reach a shelf. Is guessing enough if it has to make the trip again tomorrow?

Demonstration Use the revised 15-slide deck: brief recap; motor and grip; wheel degrees; rolling before circumference; d = 2 × r and C = pi × d; both distance conversions. Let learners infer the half-turn answer. Slides 10–12 give station tasks; slide 14 is optional. Use measured kit C.

```
wheel_degrees = 180
await drive(wheel_degrees)
await signal("D")
```

Code note wheel_degrees is wheel rotation, not centimeters. Load the full program. Finish slides 1–9 by minute 14. Slides 10–15 support the existing activity, reflection and cleanup times; the optional challenge fits only within a retest block.

Week 2 facilitation steps

1. Align a fixed mark on the chassis with the start line. Use measured circumference C to predict (180/360) × C cm. Run wheel_degrees = 180, then measure displacement of that same mark. Keep the mentor tray and cube fixed.

2. Swap driver. Predict (360/360) × C cm, then run wheel_degrees = 360. Compare the measured distance with C. Keep the same kit, floor, start direction, speed, tray and cube. Two driven wheels do not double the distance.

3. The third driver enters round((40/C) × 360) as wheel_degrees and tests the 40 cm target. Use a calculator and the kit's own measured C. Example only: C = 17.6 cm gives about 818 degrees. Do not exceed 1080; ask the mentor to check measurements if the answer is outside the allowed range.

4. During the retest, first repeat or adjust the target setting with the SAME mentor tray and cube. Record too short/too far in centimeters. Then, if time remains, swap ONLY to the team tray and repeat the same code; label this an attachment test. Do not attribute a result to code when the tray also changed.

5. Explain one reason two identical runs might differ, such as a crooked start or wheel slip. Park the robot and save the team’s best value.

Build activity A cargo tray that keeps its load. Build a removable tray or cradle with an edge that keeps the cube aboard. Choose where the cube will sit and build a flat support. Add an edge or rail without making the tray touch the floor or wheels. Tilt the tray gently by hand and compare one change to its edge. Test: Cube stays in the tray during a gentle tilt; tray does not sag into a wheel. A mentor supplies an example tray for the first robot turn. Keep payload and tray unchanged during each distance comparison; test the team tray during the retest.

Support Start with half, one and two turns. Help with calculator entry; learners still predict and explain. The wheel reference card gives both formulas and a worked example.

Optional extension After core trials, builders attach the checked cart. The instructor selects a new tabletop start and a book straight ahead, well inside all edges. Teams measure the gap from the frontmost cart/robot part, subtract a chosen clearance (initially 3 cm), and compute wheel_degrees = round((travel / C) × 360). The programmer edits that value; keep SPEED = 200 and 0–1080 degrees. Mentor checks before running; reset the same start for one reasoned correction. The robot cannot sense the book or table edges. Skip if space or time is insufficient; stop by minute 50. See slide 14 notes.

Troubleshooting Distance inconsistent: reset to the same front edge and check wheel friction. Robot arcs: check that tires match and both motors connect correctly. A target miss is data, not a coding failure.

Word Blocks equivalent Set variable travel to 0.5 rotations; move straight for travel rotations; display D. Try 1 rotation, then estimate the target. Block speed is a percentage and Python velocity is degrees per second, so do not copy speed numbers between them.

Evidence to keep Three rows: wheel degrees, prediction in cm, measured cm, error from 40 cm. Individual exit: identify the variable and defend one proposed change.

Expected reasoning One wheel turn is 360 degrees and predicts one circumference of forward travel without slip. To go backward from a target to code, divide target distance by C and multiply by 360. The motor turn sensor does not measure floor distance.

Week 3 Robot dance studio

Mission A loop repeats a routine

Learning objective Each student traces a counted loop and explains which actions repeat and how many times.

Minimum success A short routine repeats three times, stays within the test area, and the team accurately predicts the number of movements.

Materials Base, 100 × 100 cm clear floor area, action cards, pencil, student worksheet.

Before students arrive Open W03_dance.py. Test a small pivot of 90 wheel degrees before class. Use low speed and a clear perimeter. The turn helper is an open-loop pivot, not a command for a 90-degree heading change. Rehearse the straight cone recipe and target placement; pack six matching small cones and four pusher build packs.

Time	Activity
0–5	Welcome and quick check
5–10	Robot demonstration
10–14	Code and prediction
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt How could we teach a robot a dance without writing the same lines over and over?

Demonstration Compare three copies of two actions with one for loop. Say: “The indented lines belong to the loop.” Trace with a finger three times. Keep the support functions collapsed if the editor allows it.

```
for beat in range(3):
    await drive(90)
    await turn(90)
await signal("D")
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 3 facilitation steps

1. Walk the planned routine using paper arrows. Check that each repeat has the same actions. Write the predicted number of drives and turns.

2. Run the supplied three-repeat routine. Navigator counts completed repeats aloud while remaining outside the lane.

3. Swap roles. Change only the repeat count to 2. Predict and test. Return to 3 if the smaller routine is too brief to observe.

4. Design a routine using two or three commands inside the loop. Use 60–120 wheel degrees per movement and 2–3 repeats. Mentor checks the predicted area before running.

5. Use the 38–50 minute retests for the approved cone pusher and straight loop below. Predict three drives, three pauses and one D; record the cone result. Preserve driver turns and minute-50 reflection.

Build activity A removable cone pusher. Build a low front bar with two mounting points. Hand-push an upright cone 10 cm, three times, using the mounting proxy. Change height, width or a shallow V to reduce tipping or sideways escape. Keep the cone on the floor; nothing may drag or touch the wheels. Success: cone stays upright with its whole base in the target zone. Keep all attempts and compare one design change. At the stopped robot, a mentor checks clearance and fit. For the cone retest, replace the recipe with three repeats of drive(60) and pause(300), then signal("D") outside the loop. Use await before each command; omit turns. Keep SPEED = 200. Mentor marks a target suited to the rehearsed travel. Stop if the cone falls or escapes; reset only after motion stops. Remove the pusher before a dance run.

Support Provide the complete loop and allow only the number in range to change. Trace with three counters moved one at a time.

Optional extension Define a short async dance_move function and call it twice, or calibrate a pivot for an approximate corner. Keep this optional and explain the difference between wheel and heading degrees.

Troubleshooting Only one action repeats: check indentation. SyntaxError after range: check the colon. Dance drifts: reduce travel and recheck the start area; do not chase perfect geometry in this session.

Word Blocks equivalent When program starts; repeat 3: move straight 0.25 rotations, pivot using a mentor-tested tank or steering movement; after the repeat, display D. Match action order and repetition rather than numeric speed settings.

Evidence to keep Record repeats, actions inside the loop, predicted movement count, and observed count. Mark the loop body with a bracket.

Expected reasoning Three drives and three turns. D appears once because it is outside the loop. Moving D inside makes it happen on every pass.

Week 4 Parking assistant

Mission A distance reading changes a decision

Learning objective Each student predicts a threshold decision and distinguishes a valid distance reading from an invalid reading.

Minimum success The robot approaches a broad cardboard wall in small steps and stops around 150 mm away. The team explains the threshold, invalid-reading stop, and movement limit.

Materials Distance sensor on C facing forward, rigid removable bracket, broad cardboard wall, ruler, lane at least 80 cm long.

Before students arrive Mount the distance sensor before class. Open W04_parking.py. At 100, 150, and 250 mm, check readings against the ruler. Start about 300 mm from a broad, upright wall. Keep hands out of the lane.

Time	Activity
0–5	Welcome and quick check
5–10	Robot demonstration
10–14	Code and prediction
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt How can a robot decide when it is close enough without someone shouting stop?

Demonstration Show a live distance reading while the robot stays still. Python readings use millimeters. Trace the condition 0 <= gap <= stop_mm. An invalid negative value must stop the activity for a check.

```
stop_mm = 150
gap = distance_sensor.distance(port.C)
if 0 <= gap <= stop_mm:
    await signal("P")
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 4 facilitation steps

1. With wheels still, measure the wall distance using a ruler. Compare two sensor readings with the measured gap. Explain millimeters versus centimeters.

2. Set stop_mm = 150. Start roughly 300 mm away and run. The supplied helper checks the sensor before each short 30-wheel-degree movement and limits the run to 12 steps.

3. Measure the stopped distance and record the reason printed by the program: near, invalid, or limit. A limit message does not mean successful parking.

4. Swap roles. Try a 200 mm threshold from the same start. Predict whether the robot will stop sooner or later. Keep the wall broad and square to the sensor.

5. On the final turn, repeat one chosen setting. Explain why this classroom prototype is not a real vehicle safety system.

Build activity A garage with a dependable target wall. Build a stable garage entrance and a broad, flat back wall that the distance sensor can face. Make a bay wider than the base footprint. Brace the flat back wall so a light tap does not tip it over. Move the unpowered proxy through the entrance. Mark a stopping zone in front of the wall. Test: Proxy clears the entrance; target wall stays flat and upright at sensor height. Use the mentor wall during initial parking tests. The mentor approves the team garage before its robot retest; keep the broad wall and sensor line of sight clear.

Support Provide the decision rule in words and highlight the threshold number. A stationary sensor demonstration can meet the decision objective if movement fails.

Optional extension Change one wall property, such as angle, and compare valid readings. Explain why poor readings require a fallback. Do not remove the step limit.

Troubleshooting Invalid reading: check the C port and wall angle. Reaches step limit: move start closer or inspect readings. Stops too close: slow down and inspect step size. No collision is required to test the sensor.

Word Blocks equivalent Set near false. Repeat up to 12 checks: if distance is within a valid near range, set near true; otherwise, if still allowed, move straight a small increment. Use app distance units shown on the block, commonly cm: 15 cm equals 150 mm. Mentor verifies invalid-reading behavior and uses manual stop if needed.

Evidence to keep Log threshold, start gap, stopped gap, and stop reason for two trials. Individual exit uses a new reading of 170 mm with a 150 mm threshold.

Expected reasoning 220 continues. 150 and 90 are near. -1 is invalid and must not be treated as a real near distance. At 170 with a threshold of 150, continue if a step remains.

Week 5 The floor tells the robot

Mission A dark marker identifies a delivery stop

Learning objective Each student uses measured bright and dark readings to choose a threshold and explain a repeated sensor check.

Minimum success The robot stops on a wide dark floor marker in at least one of two trials, and the team explains its chosen threshold using observations.

Materials Color sensor on D pointing down, white paper or light mat, dark matte tape at least 5 cm wide across the lane, ruler.

Before students arrive Mount sensor roughly 1–2 cm above the surface as a starting setup, then bench-adjust until bright and dark readings clearly differ. Open W05_marker.py. Put the marker about 25 cm ahead of the sensor starting position. Retest under room lighting.

Time	Activity
0–5	Welcome and quick check
5–10	Robot demonstration
10–14	Code and prediction
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt Could a strip on the floor tell a delivery robot where to stop?

Demonstration Hold the robot still over light and dark surfaces. Show reflection values, whose scale is 0–100. Choose a threshold between observed readings. This lesson detects one stop marker, not a line-following route.

```
threshold = 40
reading = color_sensor.reflection(port.D)
if reading < threshold:
    await signal("M")
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 5 facilitation steps

1. Read the stationary sensor over the light surface and dark marker. Record each twice. Choose a value between the two clusters of readings.

2. Enter that value as threshold. Place the sensor behind the marker at the marked start. Run the bounded marker routine, which checks before each short movement.

3. Observe whether the robot stops with its sensor above the tape. Record the stop reason: marker or limit. Keep a clear runout beyond the strip.

4. Swap roles and repeat from the same start. If it misses, check sensor height, threshold, tape width, and start distance in that order. Change one thing.

5. Use the final turn to test the revised setting and explain how a measured threshold differs from a guessed number.

Build activity A road with a delivery marker. Build a flat road section with one clear dark stopping band and a small delivery platform beside it. Join flat tiles without raised edges in the wheel path. Add one wide dark band across the sensor path and a delivery platform off the lane. Slide the footprint template through the route and remove catches or loose tape. Test: Flat route, one visible marker and clear wheel/sensor paths. Mentor verifies the marker is wide enough for the tested movement step. Robot teams measure the actual bright/dark readings. Use those measurements to choose the threshold; a colored decoration is not automatically a reliable marker.

Support Use a number line and ask students to choose any value clearly between dark and bright. Mentor pre-mounts the bracket and helps type the condition.

Optional extension Compare results with the room lights or surface changed. Recalibrate and explain why the old threshold may fail. An advanced student can count steps without removing the 24-step limit.

Troubleshooting Always stops: sensor starts on dark tape or threshold is too high. Never stops: tape lacks contrast, sensor is too high, threshold is too low, or 24 steps cannot reach the marker. Check a stationary reading first.

Word Blocks equivalent Use reflected light intensity on D. Set found false. Repeat 24 times: if not found, check reading; if below threshold, set found true; else move a small increment. End with a marker or limit signal. Use the same measured threshold if the block reports the same 0–100 scale.

Evidence to keep Two bright readings, two dark readings, chosen threshold, and two run outcomes. Individual exit: explain a threshold for dark 20 and bright 70.

Expected reasoning 44 lies between 12 and 76. A threshold below both readings never recognizes either as dark. For 20 and 70, 45 is a reasonable starting threshold, then test it.

Week 6 A rescue attachment

Mission A small mechanical change improves delivery

Learning objective Each student contributes a design idea and compares two attachment trials while using an existing Python function.

Minimum success A removable passive plow guides one lightweight cube toward a target without blocking wheels or the sensor. The team documents one design change and explains deliver().

Materials Base with D sensor, spare beams and pins from the same kit, one 3–4 cm foam or paper cube, tape target, two small labeled attachment trays.

Before students arrive Reserve equal spare parts for each team. Test a passive plow mount. Place the stop strip 25 cm ahead of the color sensor at the starting pose. After a dry run, mark the cube delivery zone ahead of the stopped plow. Open W06_rescue.py.

Time	Activity
0–5	Welcome and quick check
5–10	Robot demonstration
10–14	Code and prediction
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt A community center needs a robot to move a dropped supply box. What shape will keep the box from escaping sideways?

Demonstration Show a flat bumper and ask what might improve it. Point to async def deliver and await deliver(). Explain that a function gives a useful group of steps a name. The provided function reuses marker detection.

```
async def deliver():
    await signal("R")
    result = await seek_marker(threshold)
    print(result)

# inside student_recipe
await deliver()
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 6 facilitation steps

1. Sketch two possible plows. Agree on one to build. Constraints: use allocated spare parts, attach to the existing chassis, no powered gripper, and keep the sensor's view clear.

2. Build a small removable attachment while the other team tests. Mentor checks wheel clearance and lightly checks that it will not fall into the drive path.

3. At the robot turn, attach the plow and place the cube just in front. Use the previously calibrated threshold. Run deliver and record whether the cube stays controlled and reaches the target area.

4. Swap driver. Change one mechanical feature, such as guide spacing, and repeat. Keep cube, program, and start pose the same so the comparison is useful.

5. In the final turn, test the better version. Remove and store each team's attachment in its labeled tray. Explain the function name and one design tradeoff.

Build activity A rescue scoop or guide. Build a removable guide that pushes a cube without trapping it under the robot. Make a flat bumper and push the cube by hand with the proxy. Add short side guides or change one guiding edge. Compare the two versions with the same cube and hand-test path. Test: Cube moves ahead; guide clears the floor and does not cover the downward sensor. Each robot station has a mentor sample before the first rotation. Fit approved team guides for retests. Keep motor and sensor mounts unchanged.

Support Provide a simple flat plow starter and ask students to change guide width. A paper mockup can support the design explanation if building takes too long.

Optional extension Compare a flat bumper with side guides using three runs each if time permits. Compute successful deliveries out of attempts. Extra motors are outside the core activity.

Troubleshooting Cube slips: adjust side guide width. Robot stalls: lighten the attachment and remove dragging parts. Sensor misses: relocate plow pieces that block the sensor. Reset the cube consistently.

Word Blocks equivalent Create a My Block called deliver, with a start signal and the bounded marker routine from week 5. Call deliver from the start stack. If My Blocks is unfamiliar, group and label the stack, then explain the repeated task in words.

Evidence to keep A labeled design sketch, two trial results, and one sentence connecting a design change to the observed outcome. Score control, not speed.

Expected reasoning Side guides can reduce lateral escape but may increase friction or interfere with sensing. Calling deliver starts its named sequence. The function itself must be defined before the call can execute.

Week 7 Community robot mission

Mission Teams choose and test a complete job

Learning objective Each team breaks a community task into at least three ordered actions and tests the most uncertain part first.

Minimum success The team has a mission plan, one saved program, one tested subsystem, and a realistic demo goal for week 8.

Materials Base, team attachment, paper cube, cardboard wall or dark stop marker, tape, blueprint worksheet, saved team projects.

Before students arrive Keep the same base and ports. Offer two core routes: supply delivery to a marker or careful parking near a shelf. Open W07_mission.py. Reserve a marked 100 × 100 cm area per kit. Each team gets its own saved recipe.

Time	Activity
0–5	Welcome and quick check
5–10	Robot demonstration
10–14	Code and prediction
14–26	First rotation
26–38	Second rotation
38–44	First retest
44–50	Second retest
50–54	Learning and observations
54–60	Save and reset

Opening prompt Who could use your robot, and what would count as a useful result?

Demonstration Model a short plan: signal ready, approach the marker, report the result. Separate required behavior from an extra feature. A turn or obstacle route is optional only after the basic task works.

```
mission = "marker"
if mission == "marker":
    result = await seek_marker(threshold)
else:
    result = await park(stop_mm)
print(result)
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 7 facilitation steps

1. Choose marker delivery or shelf parking. Name a user and write three actions in order. Agree on a measurable goal, such as two controlled deliveries in three trials.

2. Reuse the previous function for the core job. Edit the mission selector and calibrated threshold or distance. Add a start or finish signal in the recipe. Mentor checks the selected mode before running.

3. During the first robot turn, test the riskiest part alone: sensor reading, attachment, or final stop. Record the result. Each student operates the program during the turn.

4. Join the actions into one run. Keep the start mark fixed. If adding a pivot, calibrate it in an empty area first, then record the value; the default starter deliberately uses a straight route.

5. During the final turn, try the full mission and prepare a 45-second explanation: user, program decision, result, and next improvement. Save both the recipe and a copy outside the app.

Build activity A small community mission scene. Build a small scene or improve an attachment for a chosen community job. Choose a user and one job: deliver to a floor marker or stop near a garage wall. Build the matching destination, useful attachment or model of the user's need. Hand-test the scene, identify one weak feature and improve it. Test: One useful job, a clear route and a visible success condition; no new untested electronics. The robot uses one familiar tested mission. T1/T3 can test with mentor course pieces before their own scene is built. Combine the approved scene and program in the retest.

Support Offer the three-action example and a choice of only two missions. Remove extra turns. A working stationary sensor explanation is a legitimate reduced demo.

Optional extension Add a calibrated turn or combine a delivery with a parking check, keeping bounded movement. Add a custom function name and explain its input.

Troubleshooting Combined program fails: run one part at a time. Wrong team program: check team ID and save name. Mission too large: return to one sensor, one useful action, and a clear finish signal.

Word Blocks equivalent Choose the week 4 or week 5 bounded sensor stack, add a start signal and a result signal, and name the task. Students explain the same input, decision, and output as Python users.

Evidence to keep User and need, ordered plan, criterion, first test result, and planned improvement. Preserve each team's own file even though the kit is shared.

Expected reasoning There is no single mission solution. A good plan names an observable outcome and a test of its least certain behavior. All three teammates must explain a code choice.

Week 8 Robot showcase

Mission Demonstration and next steps

Learning objective Each student explains a programming idea and compares a prediction with a result in a short demonstration.

Minimum success Every team demonstrates or explains its prototype, every student completes the final assessment, and families receive an optional next-step invitation through the adult coordinator.

Materials Saved mission projects, team attachments, printed final assessment, demonstration area, optional family audience in the last 25 minutes.

Before students arrive Confirm any visitors with YMCA staff. Load team projects and inspect lanes before arrival. Use W08_showcase.py only as a clean fallback; normally reopen the team's week 7 program. No last-minute firmware update.

Time	Activity
0–7	Individual final concept check
7–12	Brief demonstration, code and prediction
12–24	Two six-minute robot rehearsals per pod; other team finishes its exhibit
24–28	Save and prepare exhibits
28–44	Four four-minute team demonstrations
44–50	Celebrate specific evidence and designs
50–54	Individual reflection and optional next steps
54–60	Save, pack and dismiss

Opening prompt What do you want your audience to understand about the decision your robot makes?

Demonstration Demonstrate a brief explanation: "We expected __. We measured __. We changed __." Show that an honest explanation of a failed run is useful engineering evidence.

```
await signal("Y")
result = await seek_marker(threshold)
print("Our result:", result)
```

Code note The code above is a discussion fragment. Run the complete supplied week file. Hardware setup and support functions are already included.

Week 8 facilitation steps

1. Complete the final four concept questions individually during minutes 0–7. These questions review concepts taught during the course; they are not a repeat of an opening test.

2. Use minutes 7–12 for a short welcome and demonstration briefing. From 12–24, each pod splits 12 minutes into two six-minute team rehearsals. Rotate the three jobs every two minutes.

3. At 24–28, save a final copy and arrange the audience safely. Each student rehearses one sentence about the code or evidence.

4. From 28–44, four teams of three each get four minutes: 45-second explanation, short run, audience question, and reset. If the run fails, show a stationary sensor reading and explain the next test.

5. At 44–50, celebrate a specific learning example from every team. At 50–54, explain high school robotics roles and the optional next step. Use 54–60 for pack-up and YMCA pickup.

Build activity A working exhibit for the showcase. Finish a stable exhibit that helps visitors understand the team's robot job. Repair or strengthen one loose part of the saved build. Add a small sign or model showing the user and useful job. Rehearse three short speaking parts while the robot is with the other team. Test: Exhibit stands, robot lane stays clear, and each child can explain one part. This is finishing and presentation time, not a new complex mechanism. Each team gets a six-minute robot rehearsal and a four-minute demonstration.

Support Allow a partner interview, pointing to code, or a recorded explanation prepared with staff. Participation and honest reasoning count even if the robot does not complete its route.

Optional extension Explain a limitation and propose a specific next experiment. Invite questions about how the result might change on a different surface.

Troubleshooting Connection failure: use the saved recipe and act out the sequence. Sensor failure: show the recorded test log. Keep each team's four-minute slot and do not replace students' explanations with a mentor repair session.

Word Blocks equivalent Reopen the team's saved Word Blocks mission. Students point out a variable, a repeat, or a decision and connect it to observed robot behavior. No language switch is needed on showcase day.

Evidence to keep Final concept responses, confidence rating, a brief mentor rubric, final recipe, and measured demo outcome. Photos or quotes require the program's agreed permission process.

Expected reasoning Use the concept-question key in the instructor guide. Report what actually happened during the demo, not the intended outcome. Future FRC interest is optional and separate from demonstrated learning.

Assessment and feedback

Week 1 opens with a welcome and robot demonstration. Prior experience was already asked about; do not repeat it or ask beginners to rate their abilities. Mentors notice support needs while children build and try the robot. Use the following questions in Week 8 after the concepts have been taught. Responses can be spoken, written or indicated with accessible choices. Record support and student IDs privately.

1 Sequence

A recipe says: show Y, drive, show D. Which action happens second?

Key Drive.

2 Variable

wheel_degrees = 180 is changed to wheel_degrees = 360. If everything else stays the same, do you predict more travel, less travel, or exactly the same travel? Why?

Key More travel is a reasonable prediction because the wheels are asked to rotate farther. Actual distance still needs testing.

3 Loop

A loop repeats drive then turn three times. How many drive actions happen?

Key Three.

4 Decision

The robot parks when a valid gap is 150 mm or less. Should it park at 100 mm? Should it park at 200 mm?

Key Park at 100 mm. Continue at 200 mm if movement is still allowed.

Scoring

Give each item 0 for no correct understanding shown, 1 for partly correct or heavily prompted, or 2 for a correct answer with an appropriate explanation. For the sequence and loop items, a correct action/count plus a brief trace earns 2. Total possible score is 8. Record the support provided. This is a locally designed formative check, not a validated standardized assessment.

Demonstration rubric

Score four dimensions separately from 0–2: program explanation, test evidence, design reasoning, and shared participation. A hardware failure does not automatically lower program explanation or participation. Use the evaluation form to describe observable behavior instead of only assigning a total.

Interpretation

Report attendance, dated examples of student work, mentor observations and the final demonstration. No opening concept test is collected, so do not calculate before-and-after score gains. Prior-experience information is context, not a substitute test score. Describe observed learning with its limits; do not claim causal proof from this small pilot.

Word Blocks and recovery guidance

The transfer that matters

Word Blocks and Python can express the same sequence, repeated action, and decision. The goal is for students to explain those ideas and connect them to the robot. Blocks do not automatically translate into the supplied Python, and numeric settings may use different units. Mentors must prebuild and test a working blocks example before using it as a class fallback.

Idea	Python recipe	Word Blocks plan
Sequence	await signal, pause, drive	Start event followed by light, wait, movement
Variable	wheel_degrees = 180	Set a variable in rotations, then use it in a movement block
Loop	for beat in range(3)	Repeat 3 around the action stack
Decision	if reading < threshold	If comparison is true, perform the action
Function	async def deliver and await deliver()	My Block deliver and its call

Bounded sensor search in blocks

Set found to false. Repeat 12 checks for distance or 24 for reflection. Inside the repeat, act only while found is false: read the sensor, evaluate the threshold, and either set found to true or move a small increment. After the repeat, show a target or limit result. For distance, add a separate valid-reading check based on what the actual app reports. If an invalid state cannot be distinguished reliably in the blocks setup, use a stationary sensor demonstration rather than autonomous movement.

Units to check

Python drive uses wheel degrees, so 360 degrees equals one wheel rotation. Python velocity uses degrees per second. Block speed normally uses percent. The Python distance sensor reports millimeters; check the displayed block unit and convert 150 mm to 15 cm when appropriate. Reflected light uses a 0–100 scale. A wheel rotation or pivot is not the robot’s heading in degrees.

Recovery order

Stop the robot. Read the error or stop reason. Check hub identity and cables. Return to the known-good file. Run a stationary light or sensor check. Retry one short movement. If the connection remains unavailable, use the paper trace and explain the predicted behavior. Ask the adult lead to resolve firmware or damaged hardware after the session.

Hardware acceptance record

Complete this on the actual YMCA kits before launch. These blank fields deliberately distinguish local rehearsal from the offline checks supplied with this package.

Check	Kit A result and initials	Kit B result and initials
Date, app version, hub firmware		
Battery, cable, and project save/export		
Front label and A/B wheel motor mapping		
90 degree forward test and manual stop		
90 wheel degree pivot and clear lane		
C sensor at 100, 150, and 250 mm		
D sensor bright and dark readings		
Working threshold and sensor mount photo		
Parking near, invalid, and limit outcomes		
Marker detected and limit outcomes		
Passive plow and sensor clearance		
Complete week 7 mission and restore saved file		

Sensor invalid behavior can be checked by pointing away from a suitable target while stationary. Do not simulate faults by unplugging electronics during movement. Confirm that a lost or invalid reading leads to a controlled stop in your installed version.

_____

_____

Launch decision and remaining actions

_____

_____

Building and participation for twelve children

12 children in grades 5–6 form four teams of three: T1/T2 share Kit A and T3/T4 share Kit B. Four student instructors cover two robot stations and two build tables; a fifth or sixth can float. The adult lead oversees the room and YMCA procedures. Bring four available laptops: two remain connected to robots and two show build references. Printed build cards make the build activity work with two laptops too.

At the robot, the driver edits and starts after “clear.” The navigator reads the step, predicts, positions the stopped robot and payload, fits an approved attachment and checks the lane. The observer watches, measures and records one number, tick or phrase. Rotate all three jobs every four minutes in the first turn and every two minutes in the retest. At the build table, rotate builder, designer/parts manager and tester/reporter on the same timer. Everyone contributes hands-on work.

14–26: T1/T3 use robots and T2/T4 build. 26–38: teams switch stations. 38–44: T1/T3 retest while the other teams finish builds. 44–50: T2/T4 retest. Rotate roles at 18, 22, 30, 34, 40, 42, 46 and 48 minutes. Each team has 18 robot minutes, giving each child about six minutes as driver plus handling, observing and building. Transitions occur within these blocks.

A real activity at the second table

Place build tables away from the robot lanes, with children facing their materials. Prepare four labeled team packs and two tested mentor examples. First robot teams use the examples or bare base; all teams can test their own approved builds in the retest. Mentors demonstrate one connection, then return the parts to students. Use pre-cut card and removable tape if spare LEGO pieces are limited.

Preparation for every class

Use 01_Instructor/Class_Preparation/README.md and the matching week checklist for exact print quantities. Trace the real base footprint and approved mounting area for four clearance templates. Keep sensors, wheels, buttons and cables accessible. Fit attachments only while stopped; use hand tests away from the robot.

Hub lights

Read the light on the correct button: steady green center means powered on with a compatible OS and at least 20% battery; flashing orange center means charge; flashing red center means stop, power off and cool for 15 minutes. White center needs a compatible OS; flashing purple center means updating. Blue is the connection button: flashing means waiting for Bluetooth, steady means connected. A display letter is a program output, not a battery or connection status. See the full SPIKE Hub Color Guide.